

Pre-Training at Scale

9/16/26 CSE 585

Ruiqi Chen, Michela Marchini, Marilyn Rego, Ellie Seehorn

Scaling Llama 3 Training with Efficient Parallelism Strategies

Weiwei Chu, Xinfeng Xie, Jiecao Yu, Jie Wang, Amar Phanishayee, Chunqiang Tang, Yuchen Hao, Jianyu Huang, Mustafa Ozdal, Jun Wang, Vedanuj Goswami, Naman Goyal, Abhishek Kadian, Andrew Gu, Chris Cai, Feng Tian, Xiaodong Wang, Min Si, Pavan Balaji, Ching-Hsiang Chu, and Jongsoo Park

ISCA 2025

Four ways to parallelize Llama 3 training across GPUs

The model and training workload are too large for one GPU, so both are divided across many GPUs.

Fully Sharded Data Parallel (FSDP)

Splits training examples and model across GPUs

Tensor Parallelism (TP)

Splits computation with each transformer layer.

Pipeline Parallelism (PP)

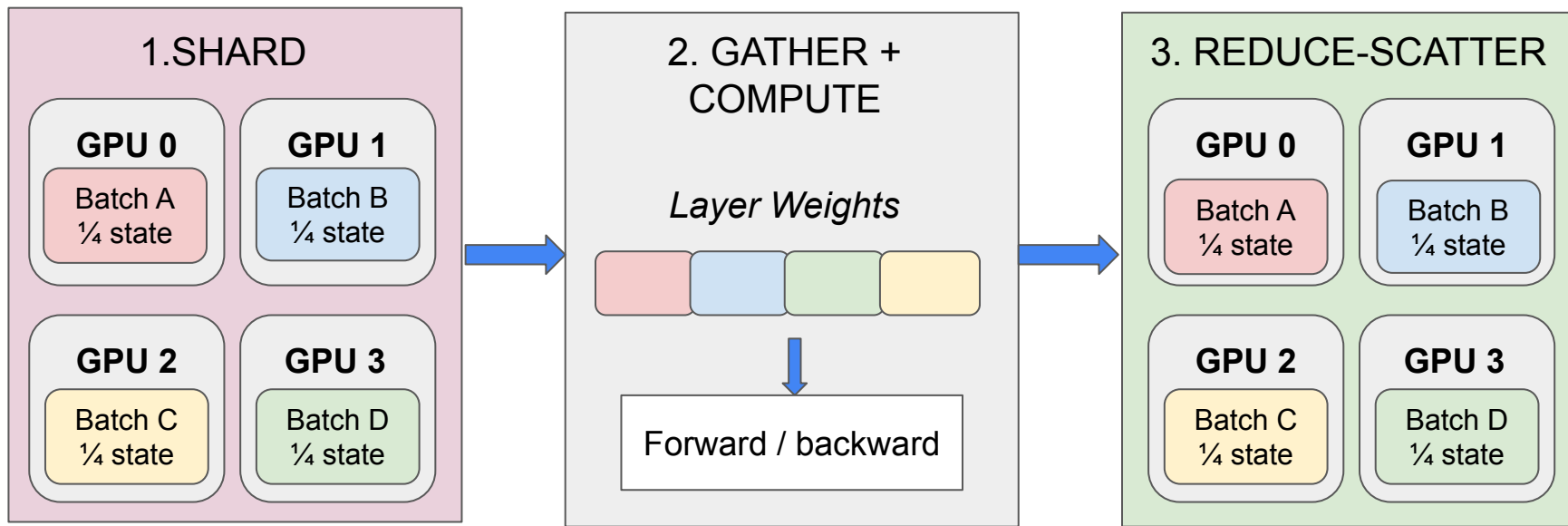
Splits consecutive group of layers into pipeline stages

Context Parallelism (CP)

Splits the tokens of one long sequence across GPUs

FSDP shards model states across GPUs

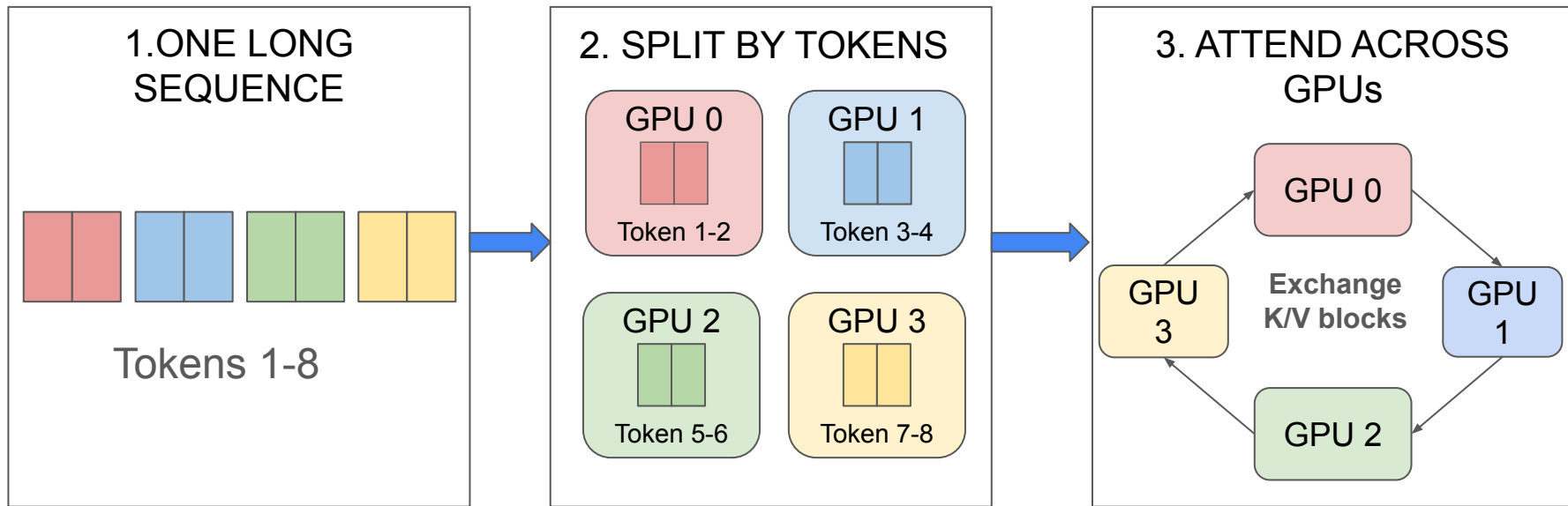
Each GPU trains on different examples, but stores only part of the model state.



Result: Parameters, gradients, and optimizer state are divided across GPUs, reducing memory, per GPU

Context parallelism splits one sequence across GPUs

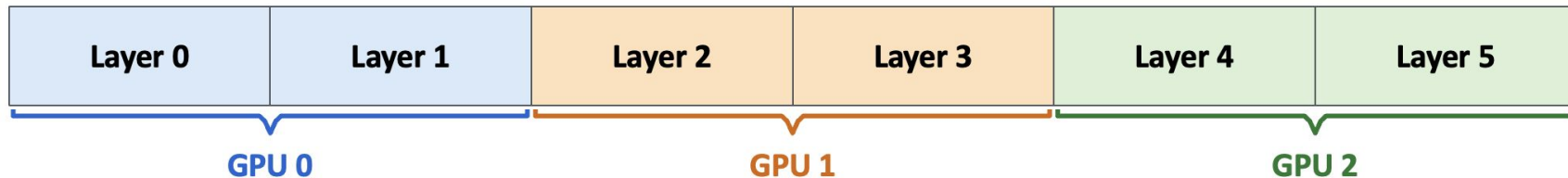
Unlike data parallelism, GPUs process different tokens from the same long sequence.



Result: Each GPU stores activation for fewer tokens, enabling much longer, sequences.

Virtual Stages enable interleaved 1F1B

Standard 1F1B: one model continuous chunk per GPU



Each GPU owns one contiguous two-layer chunk

Interleaved 1F1B: two virtual stages per GPU

Virtual stage = a smaller model chunk on an existing GPU



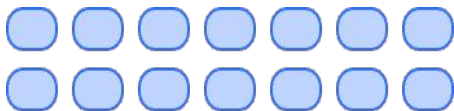
More microbatches amortize the same bubble. Virtual stages make the bubble itself shorter.

Longer sequences reduce the available data parallelism

Short context

More sequences

More data parallelism



Many short sequences

Long context

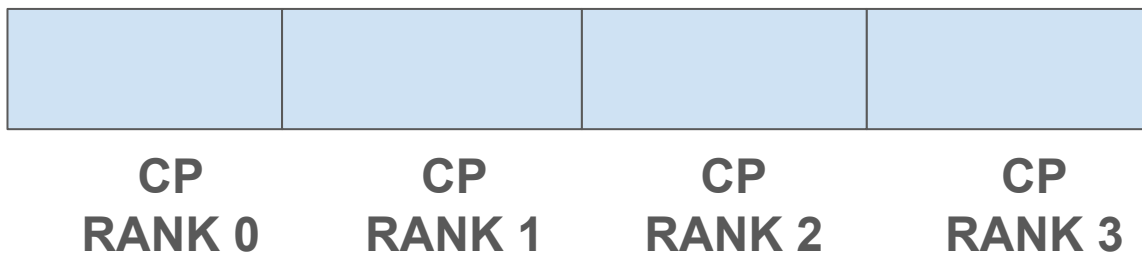
Fewer sequences

Less data parallelism



Fewer long sequences

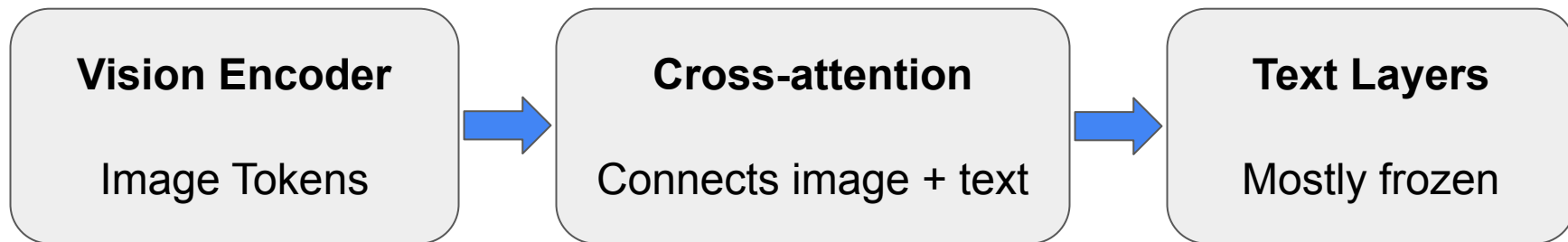
Context Parallelism splits one long sequence across GPUs



The ranks exchange the information required for attention, while each stores only part of the sequence activations.

Multimodal training creates uneven pipeline stages

A multimodal model combines components with **different compute and memory costs**



Equal layer counts can still produce unequal stage times

Pipeline and Model Codesign

Flexible 1F1B Schedule

- Relaxes micro-batch rules
- Runs extra micro-batches where needed
- Minimizes pipeline bubbles

Model-Pipeline Codesign

- Trim layer from first PP rank to save on memory
- Trim layer from last PP rank to save on head attention compute.
- Eliminate need to recompute activation

Phase Adaptive 4D Topology

- Hierarchical mapping:
(TP (NVLink) → CP → PP → DP)
- Shift DP into CP for long-context (e.g. 128k)

All-Gather Context Parallelism

Challenge: Irregular Document Boundaries

- Multiple documents in 128k context
- Document boundaries are irregular, vary by batch
- Standard ring attention cannot both manage all this context and shorten when needed
 - Can't tell document boundaries apart

Solution: All Gather CP Variant

- Bulk All-Gather of KV tensors upfront
- Pads local query chunks with leading 0s
- Allow ranks to evaluate document masks independently

Longer context is not just a cluster size problem!

To

1) maximize hardware utilization

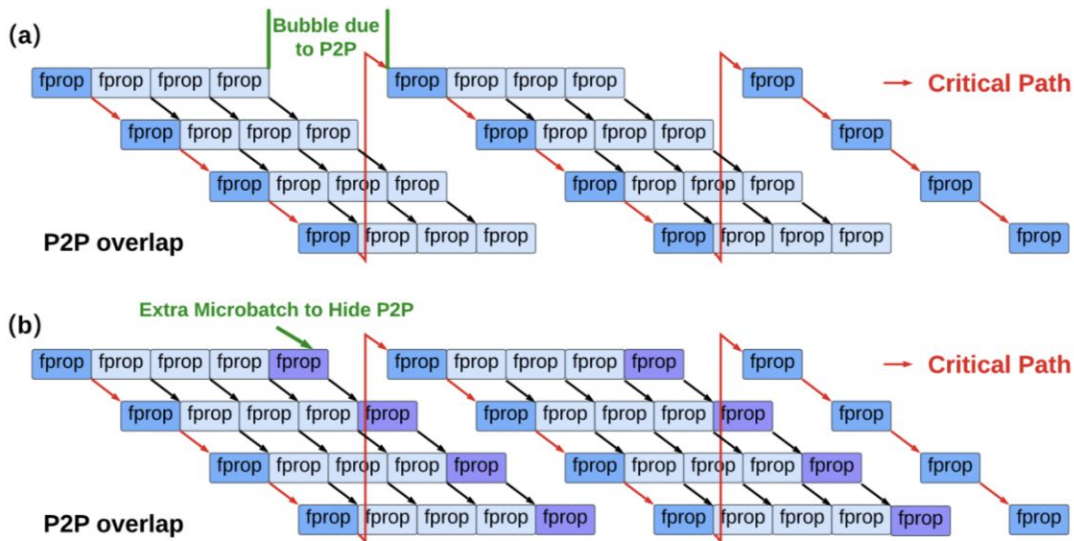
and

2) avoid hitting memory bottlenecks...

**you must co-design parallelism with model
hyperparameters (# layers and layer types)!**

Flexible PP Schedule

- Arbitrary batch size support
 - Remove constraints on number of consecutive micro-batches per stage (nc), pipeline size (pp), and number of micro-batches (nmb)
- Add micro-batches when $nc > pp$ to overlap point-to-point communication



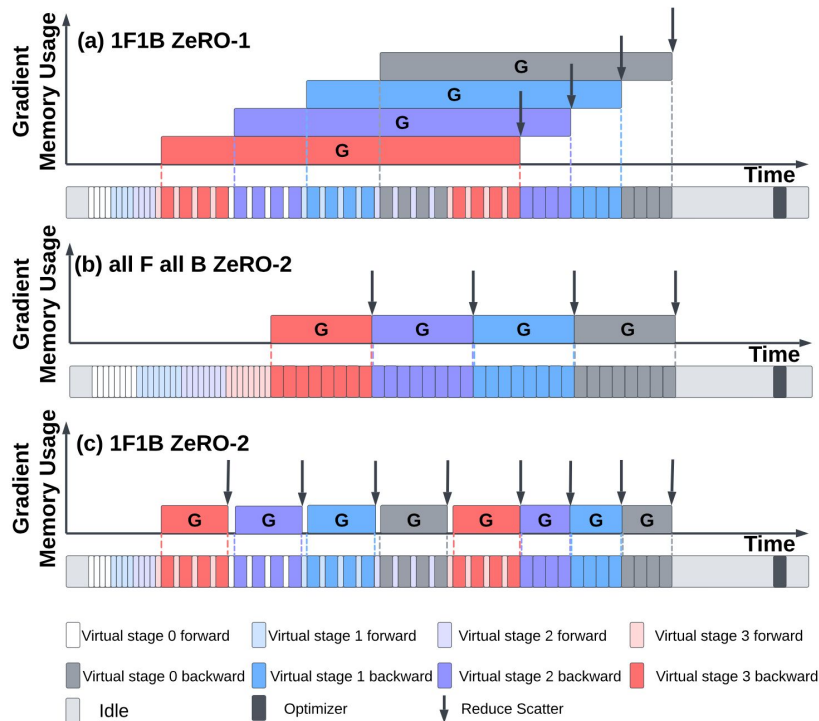
Flexible PP Schedule

When $nc > pp$

- Add $nc - pp$ warm up micro-batches per virtual stage
- Unshard gradients when batch size $\geq 2 * pp$
 - More memory, less communication overhead (FSDP Zero-1)

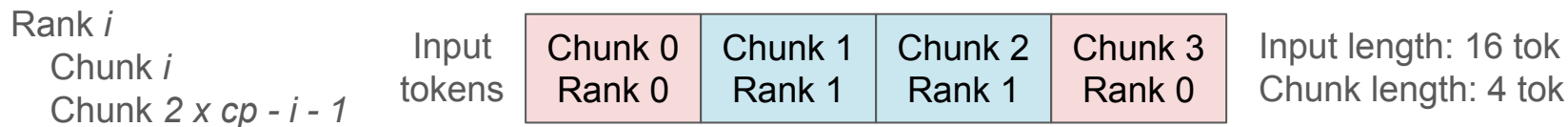
When $nc < pp$

- Schedule becomes all-forward all-backward
- Reshard gradients when batch size $< 2 * pp$
 - Less memory use, more gradient reduce (FSDP Zero-2)

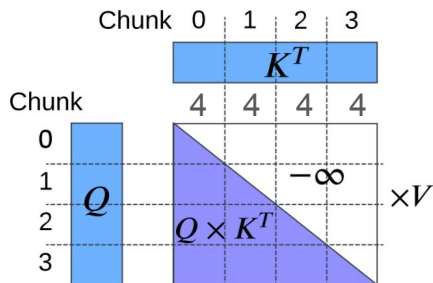


All-gather-based CP

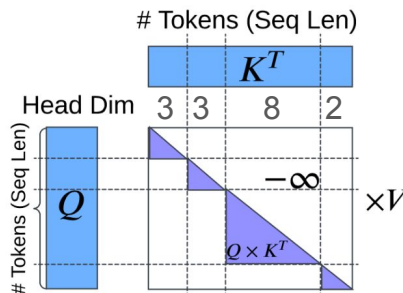
- Sharding for rank-balanced computation



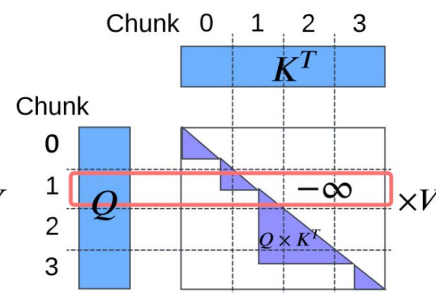
- Support irregular attention masks (document mask)
- All gather key (K) and value (V) tensors before attention computation
- Document boundary depends on input tokens, does not align with CP sharding
 - Pad Q- sequence length with leading zeros for attention computation outside chunk



(a) CP Causal Mask



(b) Doc Mask



(c) CP Doc Mask

Result 1: Achieving Peak Compute Efficiency

- Peak compute efficiency at 16k GPUs
 - **400 TFLOPs/GPU** at 8k context
 - **380 TFLOPs/GPU** at 131k long-context pretraining
 - Both over 16,384 H100 GPUs

* TFLOP: Tera Floating Point Operations (*unit measuring trillions of floating point operations*)

* TFLOPs/GPU: How many TFLOPs per second, per GPU

Result 2: Gains Achieved from Model-Pipeline Codesign

- Impact of model-pipeline codesign
 - Trimming one layer from the first PP rank (memory) and one layer from the last PP rank (compute) yields:
 - **+6.5% TFLOPs boost**
 - **+17.5% throughput gain**
 - (due to not needing activation recomputation)

Result 3: Equal Tokens $\neq$ Equal Work

- **Context parallelism accounts for 7.64% of total elapsed time at 128k context length**
- **65.75% of that exposed CP time** is idle waiting for the slowest rank to join collective barriers
 - Caused by irregular document boundaries, with non-uniform attention work

Tessera: A Holistic Pipeline Parallelism Framework for Trillion-Parameter Heterogeneous MoE Training (Operational Systems)

Weifang Hu*, Langshi Chen*, Man Yuan*, Youyang Yao, Xiulong
Yuan, Li Tian, Yong Li, Wei Lin, Xuanhua Shi, Zhengping Qian,
and Jingren Zhou

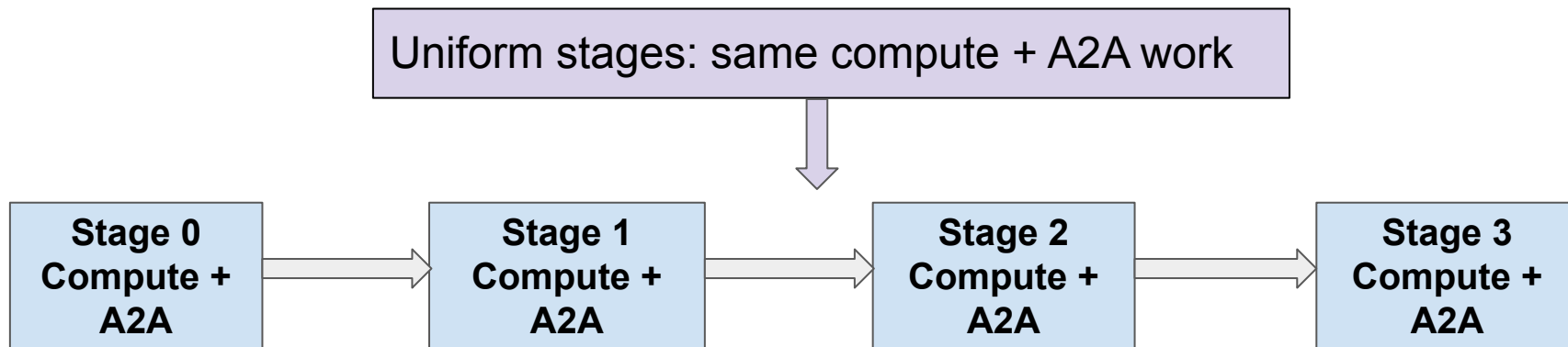
OSDI 2026

Uniform stages keep the pipeline moving at a steady rate

A balanced pipeline advances one micro-batch every stage time

Every sparse MoE layer also needs All-to-All (A2A) communication to dispatch tokens to experts and combine the outputs.

UNIFORM MoE MODEL PIPELINE



Interleaved 1F1B enables cross-microbatch overlap

Recall: A physical rank owns several virtual stages, so adjacent work on the rank can come from different chunks and micro-batches

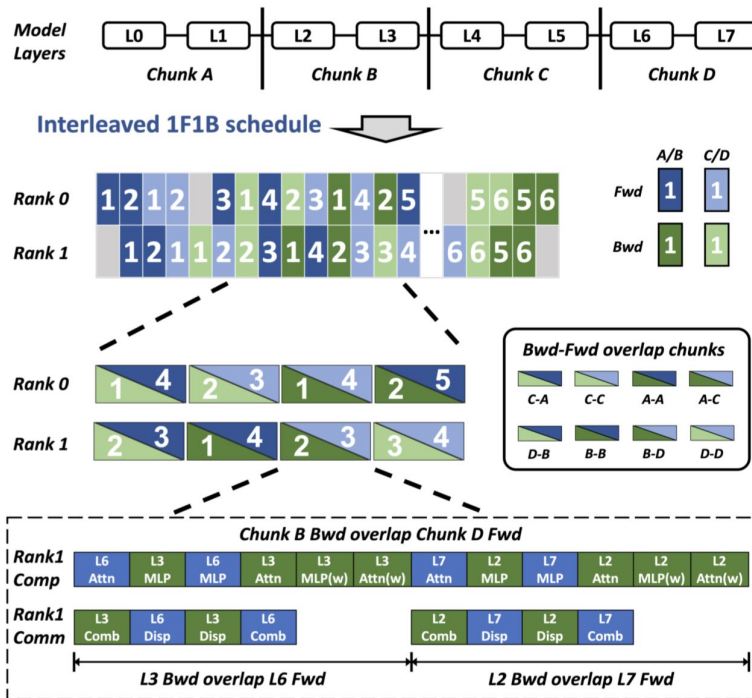
Overlap Pair

Two chunk operations that can run concurrently, communication from one can run with computation from another,

Post-overlap cost

The elapsed time that remains after the communication and computation have run concurrently

Example: Chunk B backward overlaps with Chunk D forward



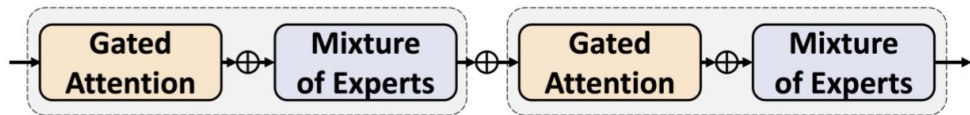
Qwen3-Next breaks the uniform-stage assumption

Earlier Assumption

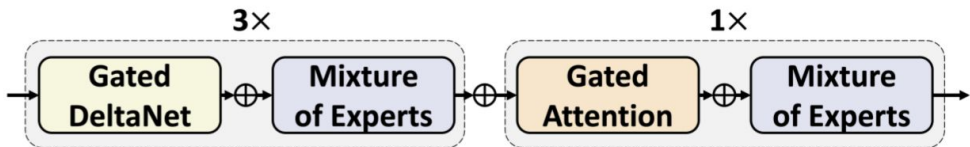
The attention and MoE pattern repeats across the model. Similar chunks have **similar compute and communication costs**.

Qwen3-Next

Gated DeltaNet and standard attention are interleaved. **Their compute profiles differ**, especially for longer sequences



(a) Homogeneous MoE Architecture



(b) Qwen3-Next Architecture

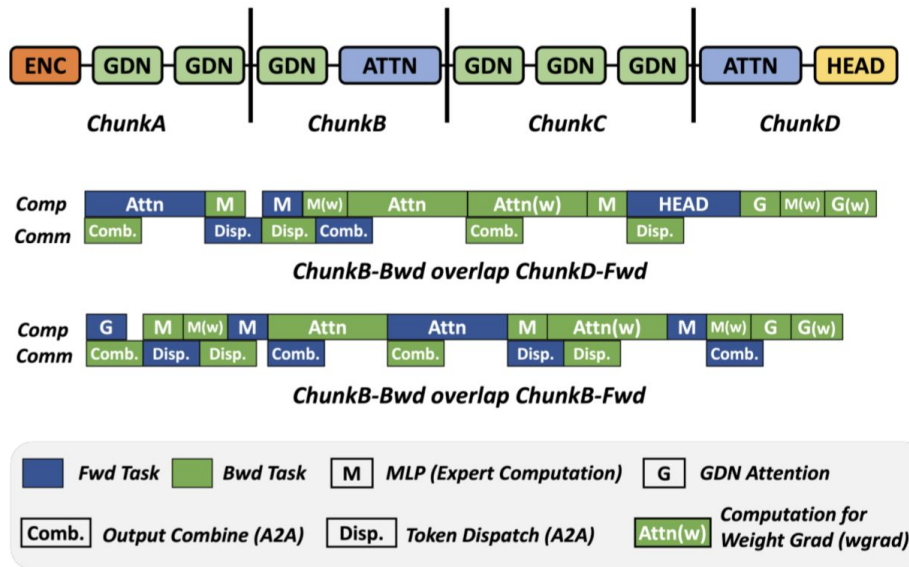
One overlap template cannot fit every chunk pair

Heterogeneous Model

Chunk composition and pass direction
change the compute-communication
timeline.

PROBLEM

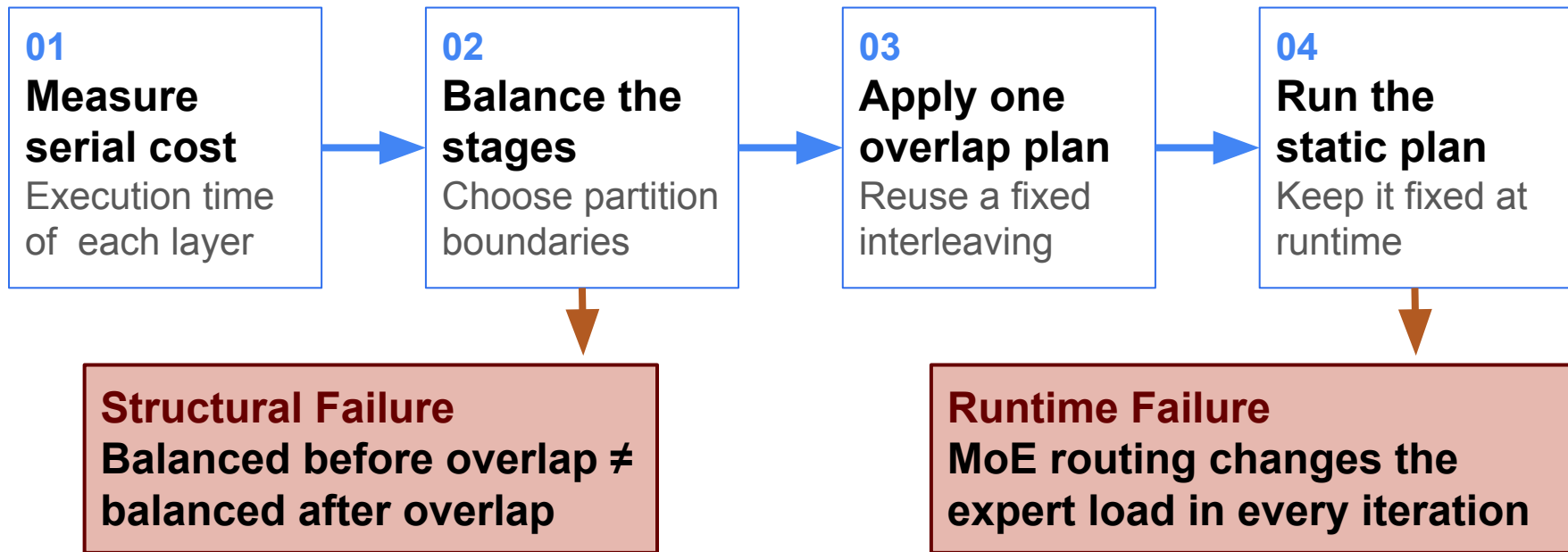
A schedule that hides A2A well for
1 pair can expose it for another



Same pipeline, different pair-specific
schedules

Why sequential pipeline planning becomes inconsistent

Existing systems make each decision once, in order



The first problem needs **joint planning**. The second needs **runtime adaptation**.

Profile Driven Co-optimization and Dynamic Adaptation

Phase 1: Profile then Partition

- Static
- Serial time $\neq$ Post-overlap time
- Profile real hardware cost and use MILP to balance **after** overlap

* MILP: Mixed-Integer Linear Programming

Phase 2: Dynamic Bubble Optimizer (DBO)

- Dynamic
- Expert token loads fluctuate step-to-step, leaving random idle gaps
- Predicts gaps ahead of time and slips in movable tasks

Profiling parallel post-overlap costs

+

Dynamically filling short idle spots on hardware (routing bubbles)

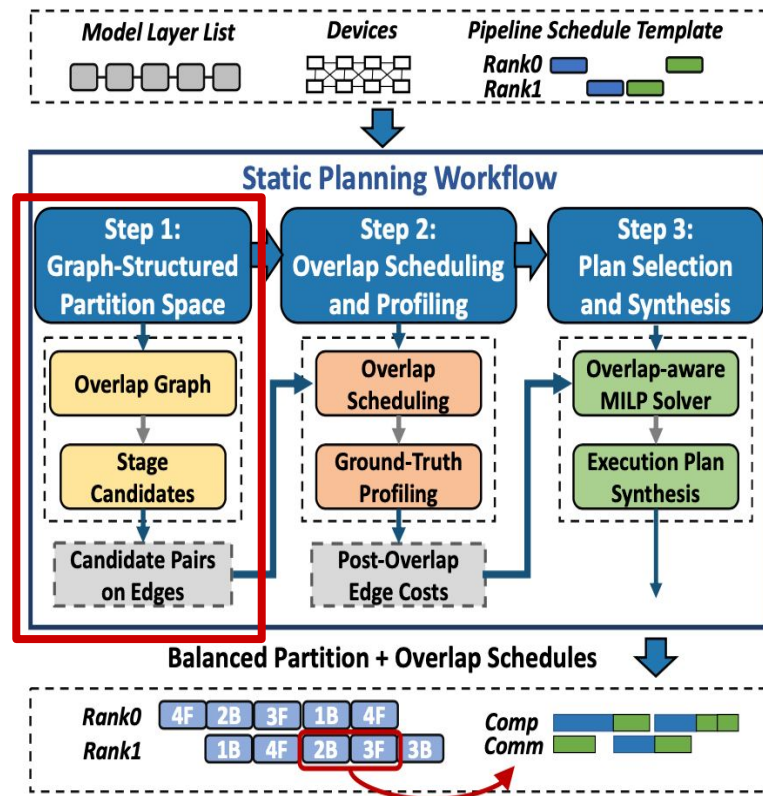
=

Reclaiming lost compute!

Static Planning

Step 1: Overlap Graph and Candidate Space

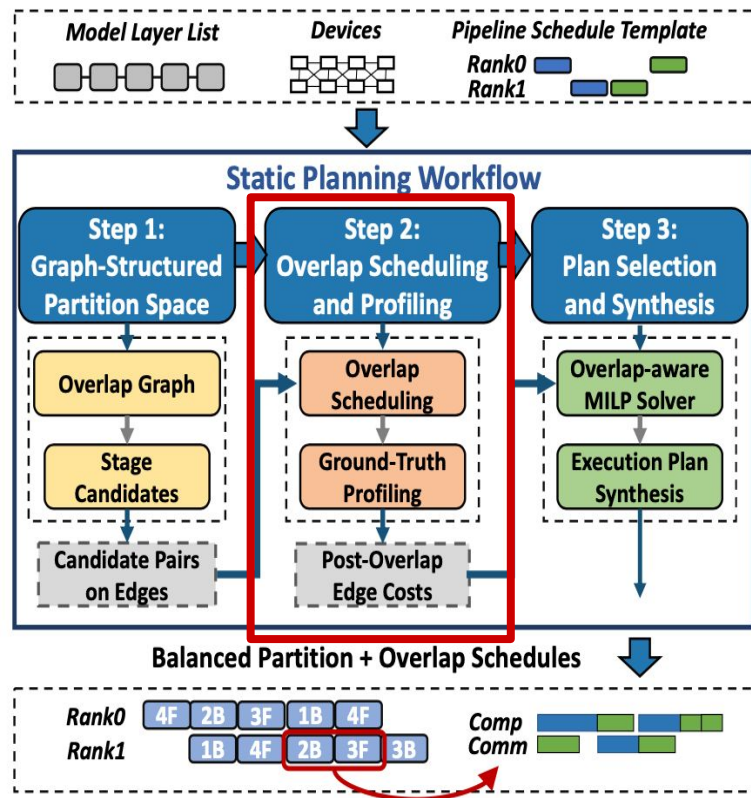
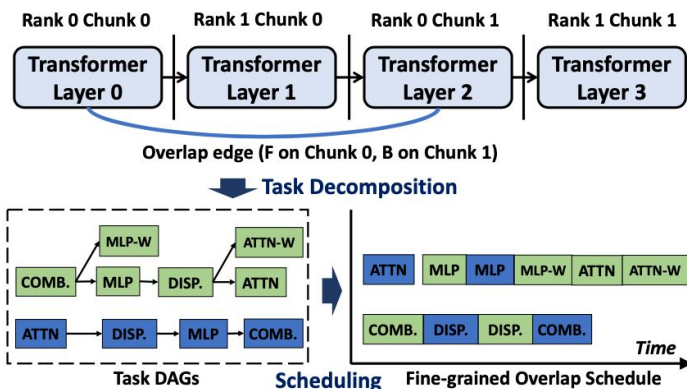
- Define search space for joint optimization
- Overlap graph $G=(S,E)$
 - Nodes S are virtual pipeline stages from template
 - Each overlap edge $e=(s,t) \in E$ is an overlap opportunity the template creates on a rank
 - Initial serial-cost-balanced assignment of layers
- C_s is candidate space of model chunks for stage s from perturbing baseline
- Candidate pair (c,d) post overlap cost
 - Min: $\max(T_c, T_d)$ (perfect overlap)
 - Max: $T_c + T_d$ (zero overlap)



Static Planning

Step 2: Overlap Scheduling and Profiling

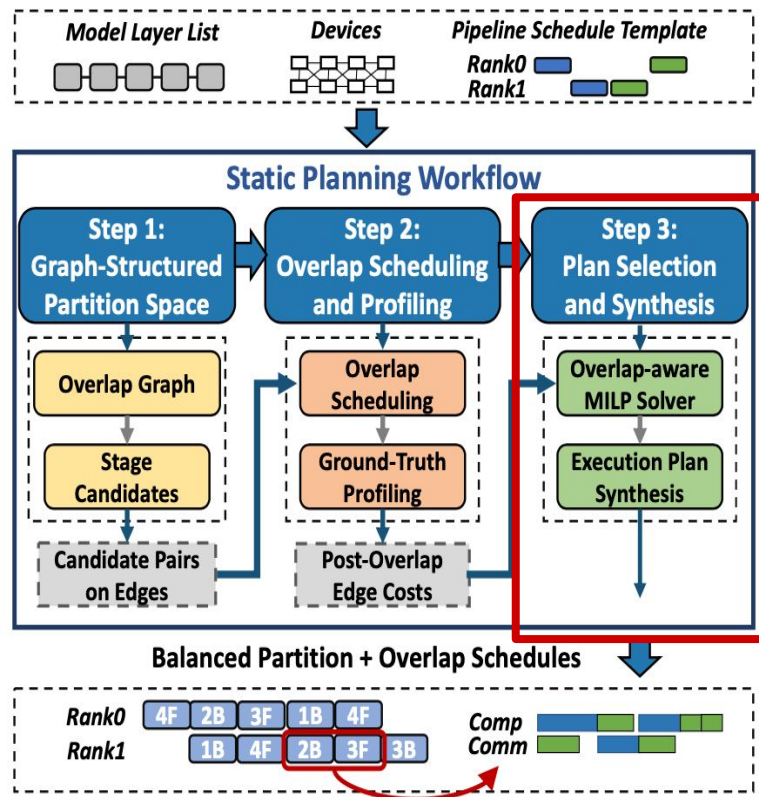
- Decompose edges into task DAGs
- Minimize pair makespans on (Comp,Comm)
 - Backbone task precedence
 - Gap-fit complementary resource
 - Movable tasks deferred then backfit
- Schedule executed on reference device group to measure ground-truth post-overlap cost



Static Planning

Step 3: Overlap Aware Partition Selection

- Partition selection reduced to graph labeling
- Maximum selected post-overlap edge cost as surrogate objective
- Plan synthesis
 - Instantiate pipeline schedule template
 - Substitute each edge with fine-grained overlap schedule from step 2



Dynamic Bubble Optimization

- Runtime variability primarily driven by token routing decisions
- Reuse routing metadata to pre-annotate predicted idle slot locations
 - Define effective insertion points for movable tasks
 - Estimate slot sizes asynchronously to overlap with useful computation
- Task injection runs immediately prior to overlapped tasks
 - Priority queues of moveable tasks based on deadline urgency
 - Guaranteed preemptive execution by deadline
 - Prioritize tight duration fit and small deadline slack

Result 1: Production Scale Gains

- Outperforms Megatron-core MoE by **1.24x MFU**
- Reaches **39% MFU** on a trillion-parameter model (Qwen3-XL)
- 20%-33% MFU improvement on 5 production workloads
 - Workloads span 4,096 to 12,288 GPUs

Result 2: Phased Component Breakdown

- Overlap aware partitioning provides **13% throughput jump**
 - *9% post overlap stage balancing, 4% A2A hiding*
- DBO adds **~5% speedup** through backfilling, at cost of 2.6 percentage points peak memory

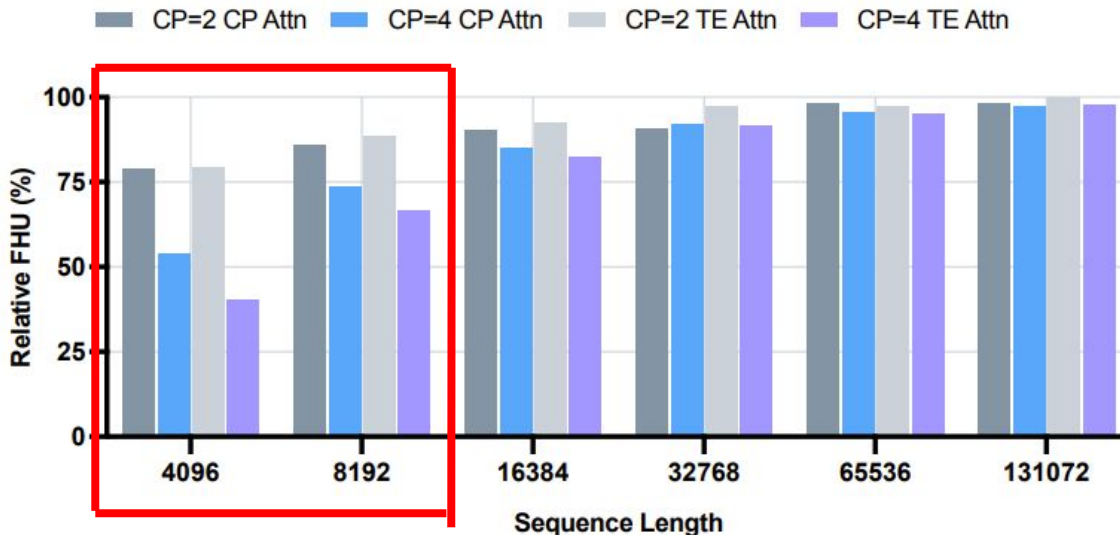
Result 3: Compute Intensity Predicts Overlap Success

- **73%** A2A communication hidden on compute heavy models (Qwen3-Next-L)
- Only **26%** A2A hidden on smaller models (Qwen3-Next-M)
 - due to shorter compute windows

Comparing Llama 3 and Tessera

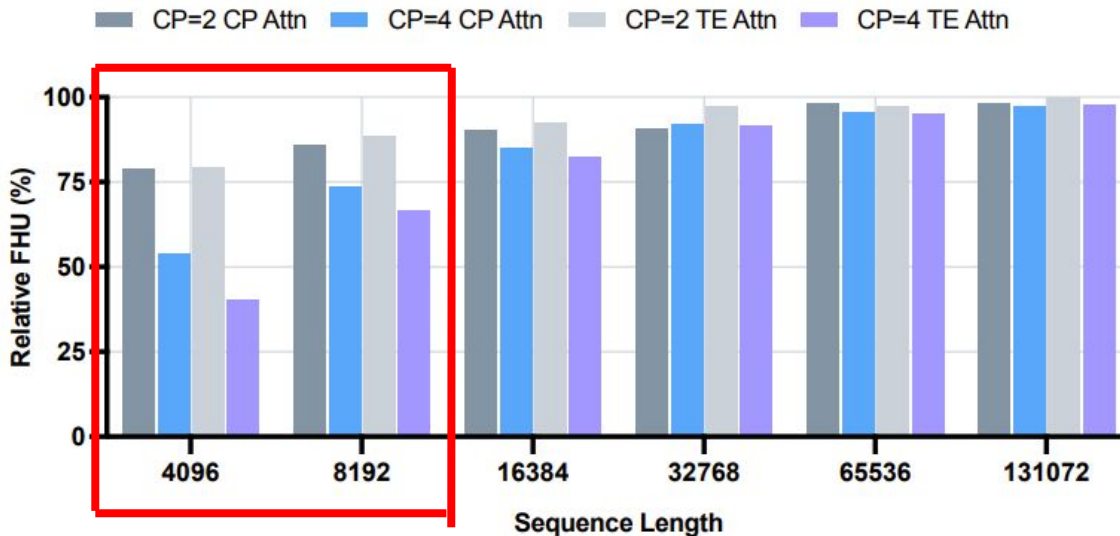
	Llama3	Tessera
<u>Target model architecture</u>	<u>Dense, uniform, transformer structures</u>	<u>Heterogeneous MOE</u>
Principal goal	Improve context length capacity without being exclusively long context	Develop method for efficiently pre-training models with mixed internal structure
<u>Partition strategy</u>	<u>Static layer trimming</u>	<u>Overlap aware MILP partitioning</u>
Communication overlap strategy	All-Gather Context Parallelism	All-to-All (A2A) Token Dispatch
Stall handling	Static rank	Dynamic bubble optimizer

Why Llama leave attention communication exposed?



More overlap does not guarantee faster attention.

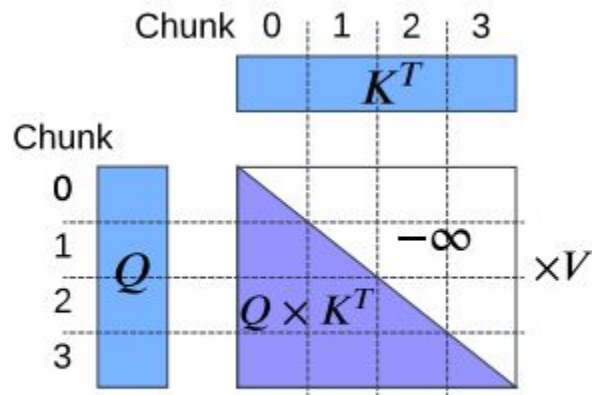
Why Llama leave attention communication exposed?



Even if this attention implementation is efficient, is the attention work itself evenly distributed across the devices?

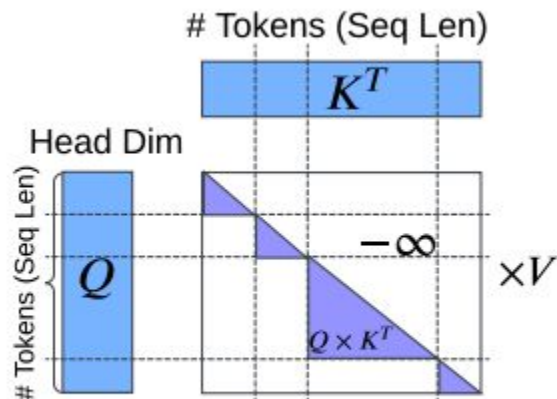
Where does equal-token sharding stop being balanced?

Full causal mask



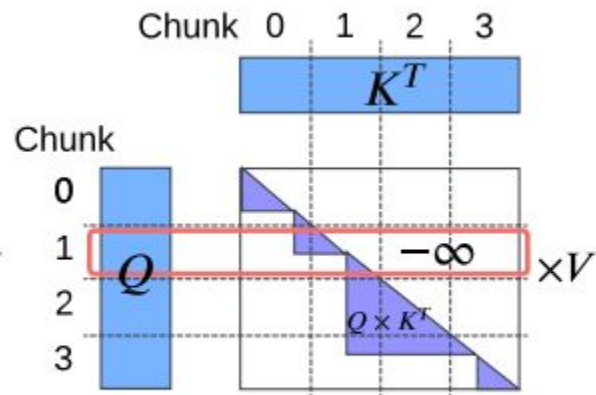
(a) CP Causal Mask

Document mask



(b) Doc Mask

Same static split

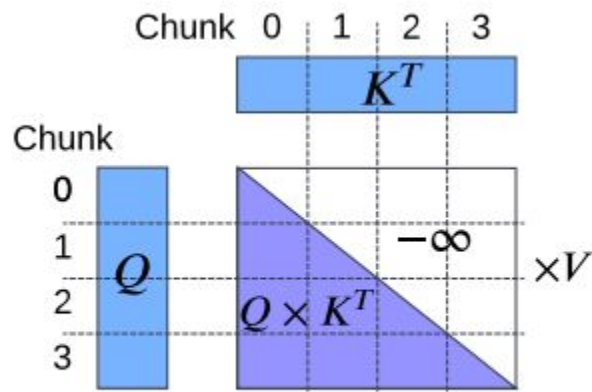


(c) CP Doc Mask

Even if every device receives the same number of tokens, does every device actually receive the same amount of attention work?

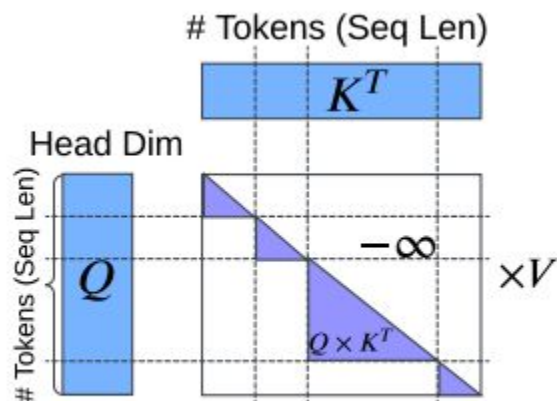
Where does equal-token sharding stop being balanced?

Full causal mask



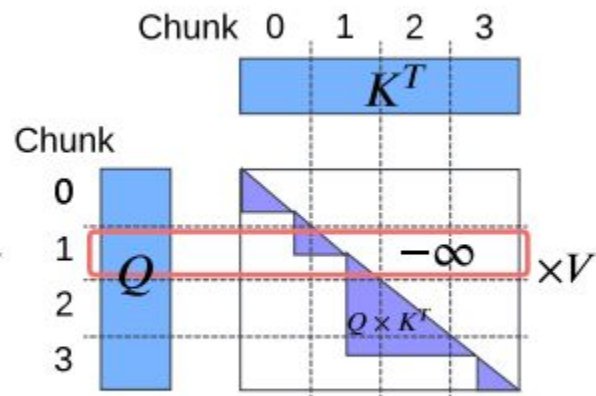
(a) CP Causal Mask

Document mask



(b) Doc Mask

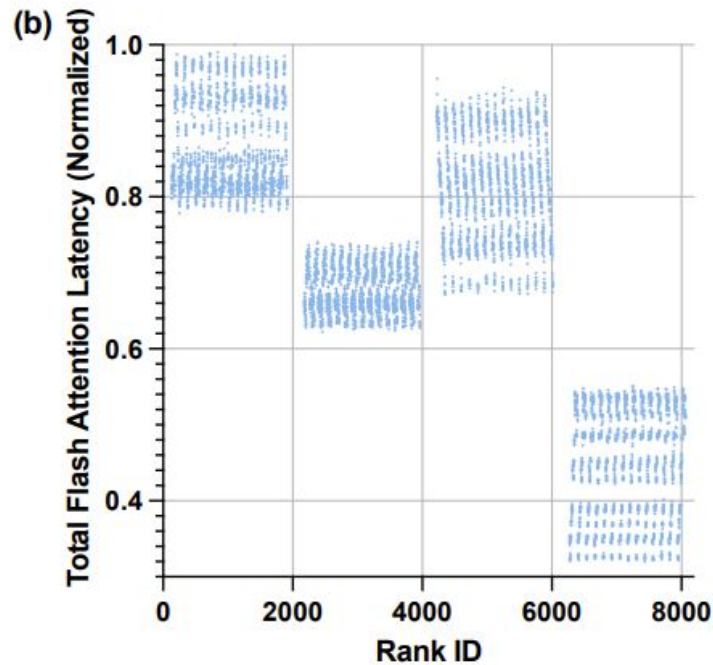
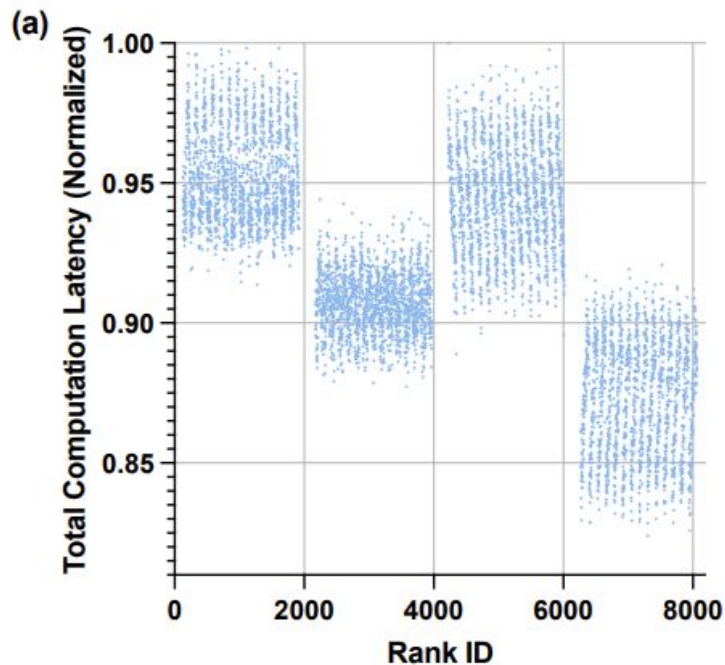
Same static split



(c) CP Doc Mask

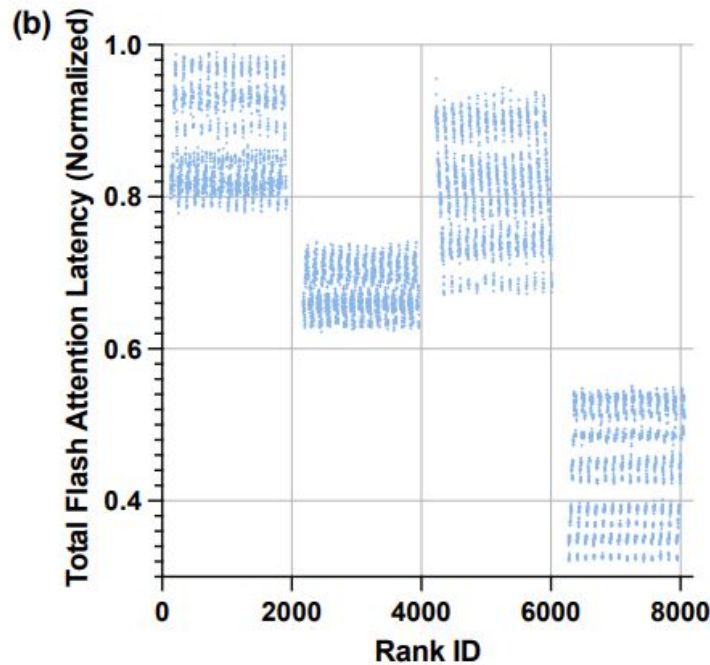
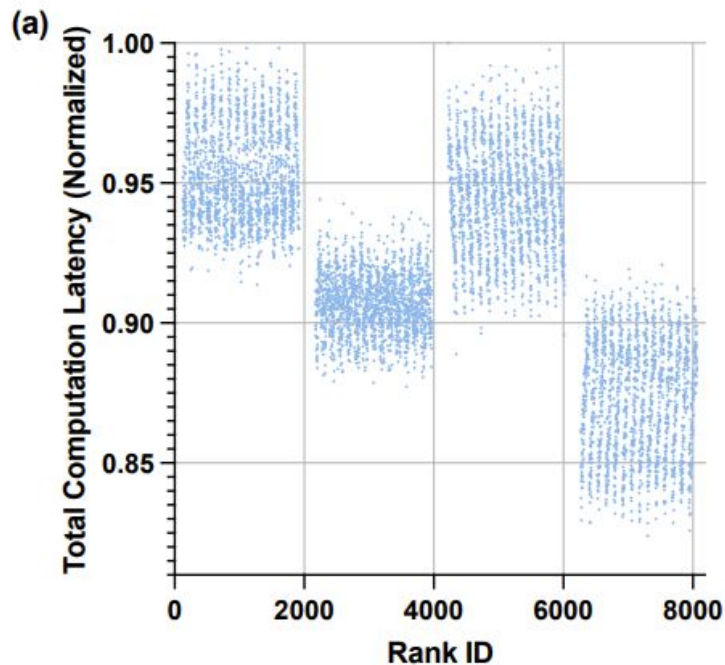
Same token count, different attention work.

What is the long collective waiting for?



The difference is coming from attention itself.

The collective is partly waiting on computation



exposed context-parallel communication takes 7.64% total elapsed time

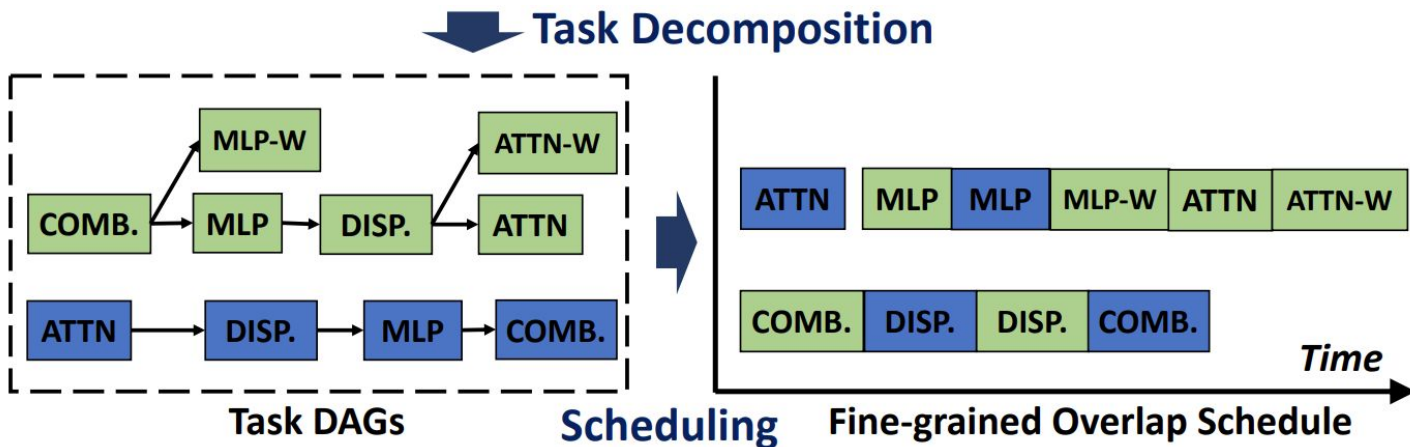
65.75% of that time: waiting for the slowest process

Remaining limitations of Llama

All-gather-based context-parallel attention design was a bad choice?

**The bottleneck moves from
how communication is implemented
to
how input-dependent work is balanced.**

Which backward work can move into a bubble?



The strength: Tessera turns real dependency slack into useful computation.

The cost: when a weight-gradient task is delayed, the activation and gradient tensors it needs must stay alive longer.

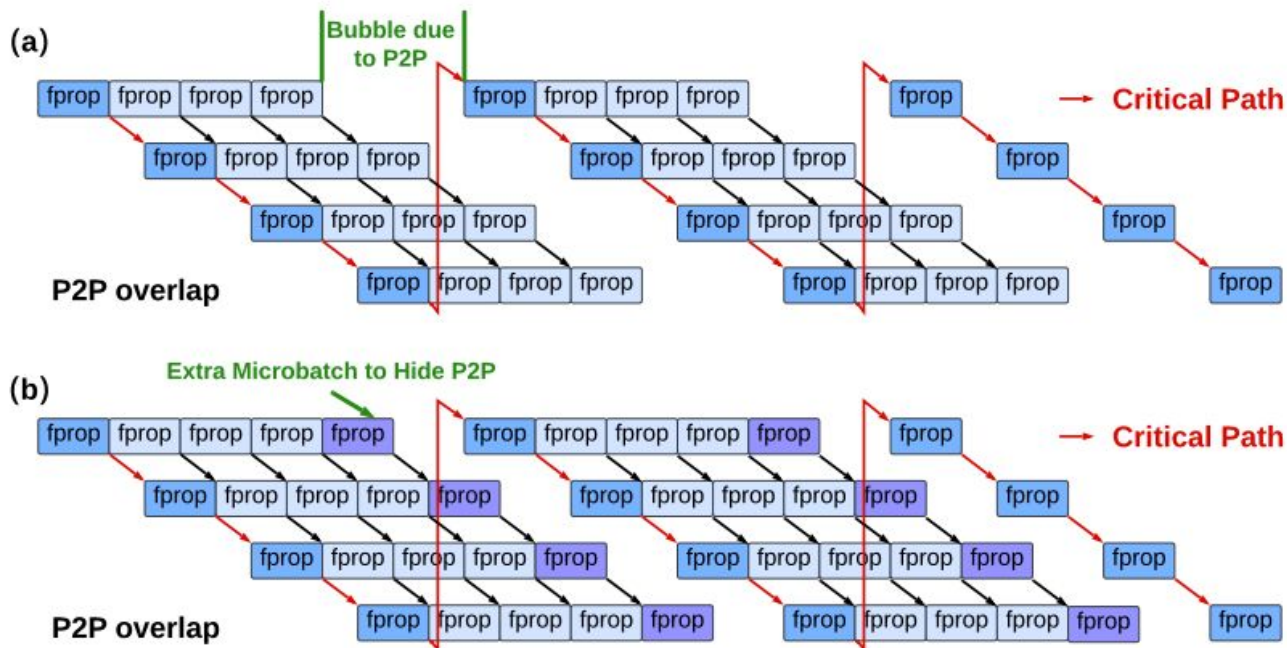
Whether that trade actually pays off?

Does task reordering pay for its overhead?

Table 3: Performance and peak-memory cost of Tessera’s Dynamic Bubble Optimizer (DBO) on Qwen3-Next-M. Peak mem. denotes the maximum allocated memory normalized by the per-GPU memory budget.

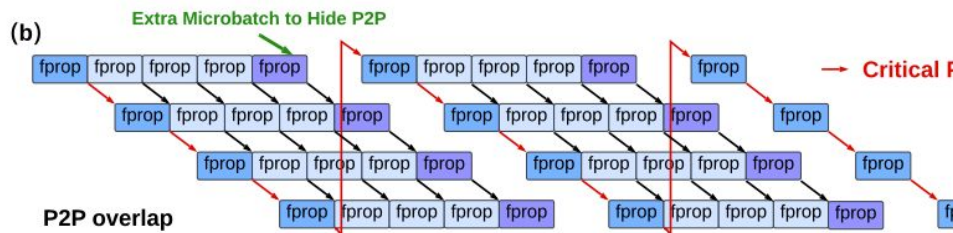
Metric	Baseline Partition	Overlap-Aware Partition
Iter-time (s), static	8.28	6.98
Iter-time (s), w/ DBO	7.83	6.67
Iter-time (s), always-keep	8.34	7.06
Peak mem., static	69.3%	67.4%
Peak mem., w/ DBO	72.9%	70.0%

More in-flight work also spends memory

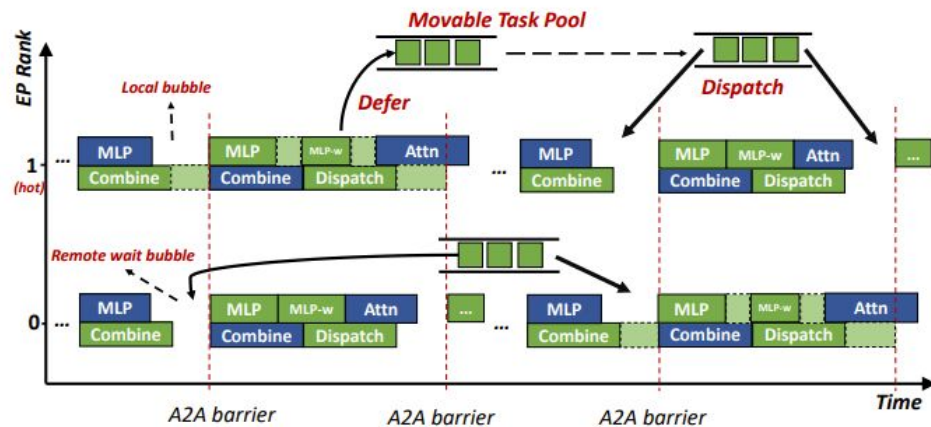


More overlap, more live activations.

Future work: co-tune the two scheduling controls together



More activations kept in memory



Task inputs kept in memory longer

One peak-memory budget

Tune separately or together?